



University of Camerino

MASTER DEGREE OF COMPUTER SCIENCE (LM-18)

Course in Knowledge Engineering and Business Intelligence

Knowledge-Based Techniques for Personalized Menu Recommendation

Authors:

Lolli Andrea 127194

andrea01.lolli@studenti.unicam.it

Berdini Angelica 127238

angelica.berdini@studenti.unicam.it

Supervisors:

Prof. Knut Hinkelmann

Prof. Emanuele Laurenzi

A.Y. 2024/2025

Index

1	Introduction	6
1.1	Project Description	6
1.2	Task Description	6
1.3	Objectives	7
2	System Architecture and Design Choices	8
2.1	System Workflow	9
2.2	Input Attributes	10
3	Knowledge-Based Solutions	12
3.1	Decision Tables	12
3.1.1	Implementation with Camunda	12
3.1.2	Migration to Trisotech	14
3.1.3	Decision Model Logic and FEEL Expressions	16
3.1.4	System Execution Examples and Results	19
3.2	Prolog	23
3.2.1	Modular and Semantic Knowledge Representation	23
3.2.2	Ontological Reasoning and Diet Constraints	26
3.2.3	Prolog-Based Execution Results	27
3.3	Ontology	30
3.3.1	Ontology structure	30
3.3.2	SWRL Rules	33
3.3.3	Individuals	35
3.3.4	SHACL Shapes and SPARQL Query	36
4	Agile and Ontology-based Meta-modelling	39
4.1	AOAME	39
4.2	BPMN 2.0	39
4.2.1	Our BPMN Model	40
4.3	Jena Fuseki	41
4.3.1	Our Jena Fuseki Query	41
5	Conclusions	49
5.1	Lolli Andrea	49
5.2	Bardini Angelica	52

Listings

3.1	Meal definitions associating meals with their course	23
3.2	Ingredients composing each meal	23
3.3	Calories associated with each meal	24
3.4	Allergens associated with each meal	24
3.5	Ingredient type classifications	25
3.6	Dietary restrictions by ingredient type	26
3.7	Diet compatibility inference rules	26
3.8	Allergy safety and calorie constraints	26
3.9	Meal recommendation rules	27
3.10	SHACL validation for Meal Shape	36
3.11	SHACL validation for Guest Shape	37

List of Figures

3.1	Decision model in Camunda	13
3.2	Meals info literal expression	13
3.3	Example of a decision table	14
3.4	Example of a row using FEEL expressions	15
3.5	Decision model in Trisotech	15
3.6	Meals info loaded in Trisotech	16
3.7	FEEL Expression - Matches Allergies	17
3.8	FEEL Expression - Matches Diet	17
3.9	Iterator - Matches Calories	18
3.10	FEEL Expression - Suitable Meals	18
3.11	FEEL Expression - Sort by Course	19
3.12	Input: No course selected, Omnivore, no allergies, no calorie restriction	19
3.13	Output: Pasta al Pomodoro, Bruschette	20
3.14	Input: Course = Appetizers, CalorieConscious = true, Omnivore, no allergies	20
3.15	Output: Bruschette al Pomodoro, Caprese	20
3.16	Input: Course = Main Dishes, DietPreference = Vegan, no allergies, no calorie restriction	21
3.17	Output: Risotto ai Funghi, Penne all'Arrabbiata	21
3.18	Input: Course = Main Dishes, DietPreference = Vegan, Allergy = gluten, no calorie restriction	22
3.19	Output: Risotto ai Funghi	22
3.20	Example 1 Output: Appetizers for Omnivore	27
3.21	Example 2 Output: Vegetarian Main Dishes	28
3.22	Example 3 Output: Meat-Based Second Courses for Carnivore	28
3.23	Example 4 Output: Gluten-Free Main Dishes	28
3.24	Example 5 Output: Gluten- and Egg-Free Desserts	29
3.25	Example 6 Output: Low-Calorie Nut-Free Vegetarian Desserts	29
3.26	Structure of the classes	30
3.27	Structure of the classes	31
3.28	CalorieConsciousMeal definition	32
3.29	Object Property: Guest hasDiet Diet	32
3.30	Data Property: Guest isCalorieConscious boolean	33
3.31	Most Restrictive Rule: celiac person, vegan and calorie conscious	34

3.32	Rule for a celiac person	34
3.33	List of Rules	34
3.34	Inferred Meal	35
3.35	Inferred rule for one guest	35
3.36	Inferred rule for one guest	36
3.37	A simple query in Protégé	38
4.1	BPMN 2.0 Model for the Meal Recommendation Process	40
4.2	Extended Task Class: RecommendMeals and its Data Properties	41
4.3	SPARQL Query for Personalized Meal Recommendation	43
4.4	Example 1 Input Parameters	44
4.5	Example 1 Meal Recommendation Output	44
4.6	Example 2 Input Parameters	45
4.7	Example 2 Meal Recommendation Output	45
4.8	Example 3 Input Parameters	46
4.9	Example 3 Meal Recommendation Output	46
4.10	Example 4 Input Parameters	47
4.11	Example 4 Meal Recommendation Output	47
4.12	Example 5 Input Parameters	48
4.13	Example 5 Meal Recommendation Output	48

1. Introduction

In the era of digitalization, many restaurants have transitioned to digitized menus accessible via QR codes. However, such menus often present challenges for users due to small screen sizes and the overwhelming number of meal options.

This project aims to enhance the digital dining experience by developing a personalized menu system that tailors meal recommendations based on guest preferences.

The solution leverages knowledge-based approaches, including decision tables, Prolog-based rule systems, and ontology-based knowledge graphs, to filter meal options according to dietary requirements, allergies, and calorie preferences. Additionally, an ontology-based meta-modeling approach is applied to extend BPMN 2.0 for meal suggestion workflows. The developed solutions are evaluated based on their effectiveness in catering to diverse user needs, highlighting the strengths and limitations of each approach.

1.1 Project Description

Many restaurants have their menus digitized. Guests can scan a QR code and have the menu presented on their smartphones. A disadvantage is that the screen is very small and it is difficult to get an overview, in particular, if the menu is large. However, some guests can not or do not want every meal, e.g. vegetarians or guests with an allergy. Instead of showing all the meals that are offered, it would be preferable to show only those meals the guest prefers. The objective of the project is to represent the knowledge about meals and guest preferences and create a system that allows to select those meals that fit the guest preferences. The knowledge base shall contain information about typical meals of an Italian restaurant, e.g. pizza, pasta, and main dishes. Meals consist of ingredients. There are different types of ingredients like meat, vegetables, fruits, or dairy. For each ingredient, there is information about the calories. Guests can be carnivores, vegetarians, calorie-conscious, or suffer from allergies, e.g. lactose or gluten intolerance.

1.2 Task Description

The specific tasks outlined for this project are as follows:

1. **Task 1:** Create different knowledge-based solutions for recommending food depending on the profile of a guest (carnivores, vegetarians, calorie-conscious, suffering from allergies, etc.) using the following representation languages:

-
- Decision tables (including DRD with sub-decisions and corresponding decision tables).
 - Prolog (including facts and rules).
 - Knowledge graph/Ontology (including rules in SWRL, queries in SPARQL, and SHACL shapes).
2. **Task 2:** Agile and ontology-based meta-modeling: Adapt BPMN 2.0 to suggest meals for a given customer. For this, the knowledge graph/ontology created in the previous task can be reused or extended. One possible approach is to specify the class BPMN Task with a new class and add additional properties, similar to what was done in class with the Business Process as a Service case. A new graphical notation should be designed for the new modeling element, making it intuitive for restaurant managers. The triple store interface (Jena Fuseki) should be used to fire the query result.

Provide a brief explanation of each solution developed. Write a conclusion chapter that discusses the advantages and disadvantages of the different knowledge-based solutions.

1.3 Objectives

This project aims to demonstrate the practical applications of knowledge engineering in a real-world scenario, by integrating various knowledge-based technologies and modeling methodologies. It also provides an opportunity to compare and evaluate the different representations, assessing their advantages and limitations in supporting personalized decision-making.

The next chapters will detail the design, implementation, and evaluation of the proposed knowledge-based solutions, described in the project description, culminating in a comparative analysis of their effectiveness and usability.

The full codebase for this project is available on GitHub at https://github.com/Andrea-Lolli/Personalized_Menu.git.

2. System Architecture and Design Choices

This chapter details the design decisions made during the development of the Personalized Menu System, focusing on its overall structure, data flow, user interaction, and considerations for scalability and user experience. Beyond functional aspects, we emphasize the non-functional requirements that guided our architecture, such as scalability, ease of onboarding for restaurant owners, privacy compliance, and maintaining a frictionless user experience.

This Personalized Menu System aims to enhance dining experiences by dynamically adjusting restaurant menus according to individual user preferences, dietary restrictions, and allergen sensitivities. Architecturally, the system we envision comprises a web-based frontend accessible via QR code scanning at restaurant tables and a backend service responsible for processing user inputs and generating personalized meal recommendations. This design ensures smooth, efficient interactions, from initial access to the delivery of a curated menu selection, while accommodating a wide range of restaurant sizes and menu complexities.

From the start, we analyzed existing solutions such as TheFork, Yelp's menu integrations, and standalone QR code menu applications. While these platforms provide visually rich experiences and basic dietary filters, few offer true personalization tailored to user-specific needs. Lastly, our system draws inspiration from a real-world restaurant menu, particularly focusing on allergen information consistent with European Union regulations. This informed our decision to incorporate allergens as a core filtering mechanism, ensuring compliance and enhancing user safety.

Key non-functional requirements influenced our design rationales:

- **Scalability:** The system is built to support deployment across restaurants of varying sizes without extensive re-engineering.
- **Ease of Onboarding:** By leveraging standard menu formats and legally mandated allergen data, restaurant owners can easily integrate their existing menus into the system.
- **User Experience:** The interface minimizes repetitive inputs and maintains clarity even on small mobile screens, ensuring users feel empowered rather than restricted.
- **Privacy Compliance:** User data handling is designed to comply with GDPR, particularly regarding the storage of preferences, maintaining user trust and legal

integrity.

2.1 System Workflow

The typical workflow begins when a user scans a QR code placed on a restaurant table. This QR code directs them to a web-based application that serves as the interface for inputting dietary preferences. Each time the QR code is scanned, the user is prompted to enter their preferences anonymously without any data being stored or cached from previous interactions. Upon accessing the interface, users specify several key parameters to tailor their menu experience. They start by selecting the type of course they are interested in, such as appetizers, main dishes, second courses, pizza, drinks, or desserts. Next, users define their guest profile by choosing from options like vegetarian (excluding meat), omnivore (no dietary restrictions), or carnivore (preference for meat-based dishes). The system is designed with scalability in mind, allowing for future integration of additional dietary profiles such as vegan, pescatarian, halal, kosher, or gluten-free without major structural changes. Allergy management is another important feature. Users select any food allergies from a predefined list of 14 allergens, consistent with European Union food regulations [Eu1], which mandate allergen disclosure on menus. If no allergens are specified, the system assumes the user has no allergy-related restrictions. Additionally, users can indicate whether they are monitoring calorie intake and, if so, set a maximum calorie threshold for their meal recommendations. After the user submits their preferences, the data is transmitted anonymously to the backend. The knowledge-based recommendation engine processes the input in real-time, filtering the restaurant’s menu according to the specified constraints. The system then returns a curated list of meal options that align with the user’s profile. For added flexibility, we suggest that the full restaurant menu is also presented alongside the personalized recommendations. This allows users to explore all available options or request modifications to dishes containing ingredients they wish to avoid.

2.2 Input Attributes

This section describes the key input parameters required by the Personalized Menu System to generate tailored meal recommendations. The inputs are designed to capture essential information about the user’s preferences and restrictions while maintaining privacy and flexibility.

- **Name and Email:** Although the interface may request the user’s name and email for identification purposes, these details are not stored to ensure user anonymity and comply with privacy considerations.
- **Course Selection:** Users specify the type of meal course they are interested in, such as appetizers, main dishes, second courses, pizza, drinks, or desserts. An “Any” option is provided to allow users without a specific course preference to receive recommendations across all courses. This feature adds flexibility but may not be available in all implementations.
- **Allergies:** The form lists common allergens aligned with European Union regulations, enabling users to easily select any allergies they have. Additionally, an “Other” field allows users to manually input any allergies not listed, supporting extensibility and accommodating a wide range of dietary needs.
- **Dietary Preference:** Users can select their dietary profile—carnivore, vegetarian, or vegan—to filter meal options accordingly. This field is optional and can be left blank to indicate an omnivore diet with no specific restrictions.
- **Calorie Consciousness:** To support calorie-aware dining, users indicate their level of calorie consciousness. In the presented example, three predefined levels are offered (e.g., low, medium, high), which serve as default calorie thresholds for filtering meals. This input accommodates users who wish to choose lighter dishes but may not know their exact calorie limits. It is worth noting that this aspect has been implemented with some variation across different versions of the system.

Below is a screenshot illustrating the input form created using Google Forms. It demonstrates how these parameters are presented to the user in a simple and intuitive way:

Guest Preferences

Descrizione modulo

Name *

Testo riaperta breve

Email

Testo riaperta breve

Select Course

☐ Appetizers
☐ Main Dishes
☐ Second Course
☐ Pizza
☐ Dessert
☐ Any

Do you suffer from some food allergies?

☐ Gluten
☐ Lactose
☐ Nickel
☐ Other

Specify other allergies (e.g. shrimp) *

Testo riaperta breve

Do you follow some particular diets?

☐ Carnivores
☐ Vegetarians
☐ Calorie-conscious
☐ Vegan

How Calorie-conscious are you?

Little
 ☐ 1
 ☐ 2
 ☐ 3
 Much

3. Knowledge-Based Solutions

In this chapter, we describe three knowledge-based approaches used to recommend meals based on the user profile and the corresponding input defined in Chapter 2. The approaches implemented are: (1) Decision Tables, (2) Logic Programming with Prolog, and (3) Ontologies using Knowledge Graphs. Each technique provides a different perspective on how structured domain knowledge can be applied to make personalized food recommendations, especially considering dietary preferences, calorie limitations, and allergens.

3.1 Decision Tables

Decision Tables are a classical method in knowledge-based systems for representing complex decision logic in a compact and readable form. A Decision Table organizes conditions and corresponding actions into a tabular format, where each row represents a rule, and each column corresponds to an input condition or an output decision. The formal foundation of Decision Tables lies in rule-based systems and logic modeling, often supported by the DMN (Decision Model and Notation) standard. DMN provides a clear separation between decision logic and business processes and is widely used for defining operational decisions in a way that is both executable and understandable to domain experts.

3.1.1 Implementation with Camunda

The initial implementation of the knowledge-based recommendation system was developed using the Camunda platform, which provides modeling tools for both BPMN (Business Process Model and Notation) and DMN (Decision Model and Notation). In this project, we focused on using Camunda's DMN engine to create a rule-based recommendation system driven by decision tables.

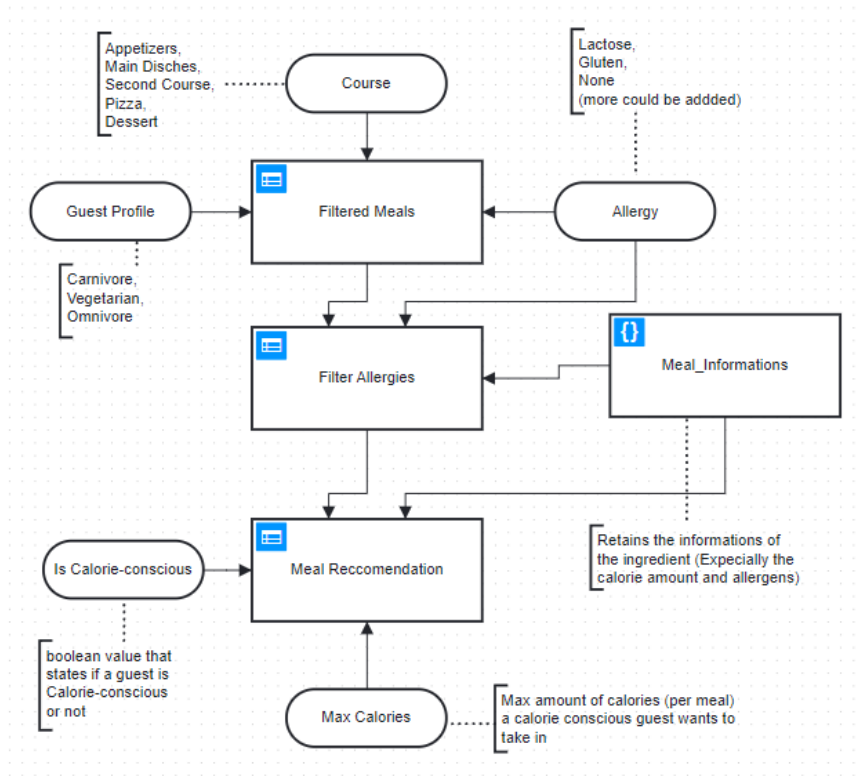


Figure 3.1: Decision model in Camunda

The system above was designed to take several user-defined parameters as input: dietary preference (e.g., vegetarian, carnivore), allergies (represented as a list of allergens), calorie sensitivity (with a maximum calorie limit), and course type (e.g., appetizer, main dish, dessert). Based on these inputs, the system filtered meals from a predefined restaurant menu using a series of decision tables.

The restaurant menu itself was implemented as a literal expression (shown below) containing all available meals, each described with relevant attributes such as name, category, type, allergens, and calories.

```

Meal_Information
[
  {
    "name": "Bruschette al Pomodoro", "calories": 180, "allergens": ["gluten", "milk"]},
    {
    "name": "Caprese", "calories": 250, "allergens": ["milk"]},
    {
    "name": "Spaghetti alla Carbonara", "calories": 565, "allergens": ["gluten", "eggs", "milk"]},
    {
    "name": "Risotto ai Funghi", "calories": 450, "allergens": ["milk", "celery"]},
    {
    "name": "Lasagna alla Bolognese", "calories": 750, "allergens": ["gluten", "milk", "eggs", "celery"]},
    {
    "name": "Penne all'Arrabbiata", "calories": 325, "allergens": ["gluten"]},
    {
    "name": "Bistecca alla Fiorentina", "calories": 655, "allergens": []},
    {
    "name": "Pollo alla Cacciatora", "calories": 570, "allergens": ["celery"]},
    {
    "name": "Melanzane alla Parmigiana", "calories": 550, "allergens": ["gluten", "milk", "eggs"]},
    {
    "name": "Margherita", "calories": 600, "allergens": ["gluten", "milk"]},
    {
    "name": "Diavola", "calories": 650, "allergens": ["gluten", "milk"]},
    {
    "name": "Prosciutto e Funghi", "calories": 700, "allergens": ["gluten", "milk"]},
    {
    "name": "Fornarina", "calories": 250, "allergens": ["gluten", "milk"]},
    {
    "name": "Gluten-free Margherita", "calories": 850, "allergens": ["milk"]},
    {
    "name": "Tiramisu", "calories": 450, "allergens": ["gluten", "eggs", "milk"]},
    {
    "name": "Panna Cotta", "calories": 250, "allergens": ["milk"]},
    {
    "name": "Biscotti", "calories": 125, "allergens": ["gluten", "milk", "eggs", "nuts"]}
]

```

Figure 3.2: Meals info litteral expression

Each decision table in the model applied a specific filtering logic based on one of the input parameters. For example, one table would filter out meals that do not match the user’s dietary preference, while another would exclude dishes containing allergens listed by the user. The filtering logic was implemented using the “collect” hit policy, which accumulates all rules that return a positive match. Each row in the decision table represents a single meal, and if it satisfies the filtering condition, it is included in the output list.

Meal Recommendation Hit policy: Collect

	Filtered Meals	And	Is Calorie-conscious	And	Meal_Information	Then	Output
	list		boolean		list		string
1	list contains(Filtered Meals, "Bruschette al Pomodoro")		true		Meal_Information[name="Bruschette al Pomodoro"].calories[1] < Max Calories		"Bruschette al Pomodoro"
2	list contains(Filtered Meals, "Caprese")		true		Meal_Information[name="Caprese"].calories[1] < Max Calories		"Caprese"
3	list contains(Filtered Meals, "Spaghetti alla Carbonara")		true		Meal_Information[name="Spaghetti alla Carbonara"].calories[1] < Max Calories		"Spaghetti alla Carbonara"
4	list contains(Filtered Meals, "Risotto ai Funghi")		true		Meal_Information[name="Risotto ai Funghi"].calories[1] < Max Calories		"Risotto ai Funghi"
5	list contains(Filtered Meals, "Lasagna alla Bolognese")		true		Meal_Information[name="Lasagna alla Bolognese"].calories[1] < Max Calories		"Lasagna alla Bolognese"
6	list contains(Filtered Meals, "Penne all'Arrabbiata")		true		Meal_Information[name="Penne all'Arrabbiata"].calories[1] < Max Calories		"Penne all'Arrabbiata"
7	-		false		-		Filtered Meals

Figure 3.3: Example of a decision table

The decision table shown above represents the final decision table in the model, which is responsible for filtering meals based on the user’s calorie preferences. Specifically, the expression illustrated evaluates each meal in the input list by checking its associated calorie value. If a meal exceeds the calorie threshold defined by the user, it is excluded from the output. However, if the parameter *isCalorieConscious* is set to false, the decision table bypasses the filtering logic and returns the original list of meals without any modifications.

The logic of the decision tables in the model relied heavily on FEEL (Friendly Enough Expression Language), a language integrated into the DMN standard for writing conditions and expressions. In particular, the model used FEEL’s advanced features for list processing, such as checking for the presence of allergens by evaluating whether any elements in the user’s allergen list were present in the dish’s allergen list. This made it possible to write expressive and compact filtering rules directly within the decision tables.

3.1.2 Migration to Trisotech

Due to limitations in Camunda’s cloud environment, specifically the discontinuation of the built-in DMN execution service, it was necessary to migrate the decision logic to the Trisotech platform. Trisotech provided a more comprehensive and user-friendly web application for modeling, executing, and testing DMN-based decisions. Despite this, we encountered several challenges in the migration. Some of the key difference between the two platforms lies in the interpretation of FEEL within the Decision Tables. While Camunda supported advanced FEEL expressions within input entries (e.g., to check whether any element in the allergen list matched an allergen in a dish), Trisotech imposes stricter syntactic constraints. Specifically, boolean FEEL expressions cannot be written directly within the cells of input columns in Trisotech tables (unless the input is a boolean value).

As a result, the filtering logic based on allergens had to be redesigned. In Camunda, a single row could include a FEEL expression such as:

	Filtered Meals	Allergy	Meal_Information	Output
	list	list	list	
1	list contains(filtered_meals, "Bruschette al Pomodoro")	is empty(Allergy) = false	is empty(duplicate values(concatenate(Allergy,Meal_Information.allergens)))	"Bruschette al Pomodoro"

Figure 3.4: Example of a row using FEEL expressions

In Trisotech, this had to be moved outside of the table logic or handled through literal expressions of iterators to apply a FEEL expression to each of the items of a list. Because of these technical differences, it was not possible to reuse the decision tables directly from the Camunda model. Nevertheless, the core logic of the original decision model was preserved and successfully re-implemented in Trisotech.

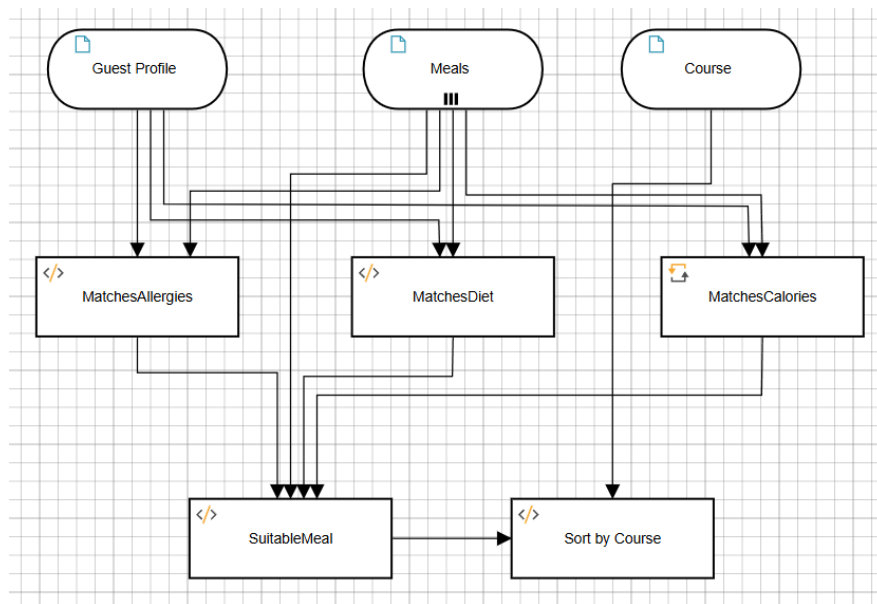


Figure 3.5: Decision model in Trisotech

The new model in Trisotech accepts as input the user's dietary profile, a list of allergens, a calorie limit (if applicable), and the type of meal (e.g., appetizer, main dish, dessert). The menu of the restaurant is loaded from an xlsx file. It then returns a filtered list of meals that satisfy all constraints.

name	ingredients	allergens	calories	course	
Bruschette al Pomodoro	tomato, bread	gluten, mi	180	Appetizers	
Caprese	tomato, chees	milk	250	Appetizers	
Spaghetti alla Carbonara	meat, eggs, oil	gluten, eg	565	Main Dishes	
Risotto ai Funghi	rice, mushroom	milk, cele	450	Main Dishes	
Lasagna alla Bolognese	meat	gluten, mi	750	Main Dishes	
Penne all'Arrabbiata	tomato, oil	gluten	325	Main Dishes	
Bistecca alla Fiorentina	meat		655	Second Course	
Pollo alla Cacciatora	meat	celery	570	Second Course	
Melanzane alla Parmigiana	eggplants, chees	gluten, mi	550	Second Course	
Margherita	tomato, wheat	gluten, mi	600	Pizza	
Diavola	tomato, meat, whe	gluten, mi	650	Pizza	
Prosciutto e Funghi	meat, tomato, whe	gluten, mi	700	Pizza	
Fornarina	wheat	gluten, mi	250	Pizza	
Gluten-free Margherita	tomato, chees	milk	850	Pizza	
Tiramisù	chees, sugar	gluten, eg	450	Dessert	
Panna Cotta	milk, boh	milk	250	Dessert	
Biscotti	wheat, sugar	gluten, mi	125	Dessert	

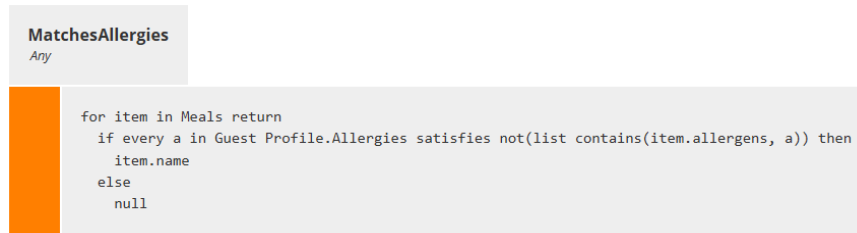
Figure 3.6: Meals info loaded in Trisotech

Additionally in our model, we defined specific data types for each input to ensure consistency and robustness in the decision logic. In details: the Course input was implemented as an enumeration with predefined values: Appetizers, Main Dishes, Second Course, Pizza, and Dessert. The Meals input is modeled as a list of custom "Meal" objects, each containing multiple fields such as name, ingredients, allergens, calories, and course. The GuestProfile variable is also a structured data type that includes three fields: dietPreference (with allowed values: Omnivore, Vegan, Vegetarian, Carnivore, Pescetarian), calorieConscious (a Boolean), and allergies (a list of strings). By using well-defined data types and enumerations, we ensure that the system behaves reliably regardless of user input, effectively eliminating input errors and enforcing valid input formats.

3.1.3 Decision Model Logic and FEEL Expressions

In this section, we explain the logic implemented in our Trisotech decision model using a series of FEEL (Friendly Enough Expression Language) expressions. These expressions are used to filter and recommend meals based on user preferences such as allergies, diet, and calorie constraints. Below, each FEEL expression is explained in the context of its role in the decision-making process.

1. Matches Allergies



The image shows a code block for a FEEL expression. On the left, there is a grey box with the text 'MatchesAllergies' and 'Any' below it. To the right of this box is a large orange vertical bar. To the right of the orange bar is a light grey box containing the following FEEL expression:

```
for item in Meals return
  if every a in Guest Profile.Allergies satisfies not(list contains(item.allergens, a)) then
    item.name
  else
    null
```

Figure 3.7: FEEL Expression - Matches Allergies

This expression filters out meals that contain ingredients a guest is allergic to. For every item in the input `Meals` list, it checks whether none of the allergens listed for that item match any of the allergies specified in the guest's profile. If the meal passes this test (i.e., it is safe for the guest), the meal's name is included in the output list; otherwise, `null` is returned.

2. Matches Diet



The image shows a code block for a FEEL expression. On the left, there is a grey box with the text 'MatchesDiet' and 'Text' below it. To the right of this box is a large orange vertical bar. To the right of the orange bar is a light grey box containing the following FEEL expression:

```
for item in Meals return
  if Guest Profile.DietPreference = "Vegan" then
    if not(list contains(item.ingredients, "meat")) and not(list contains(item.ingredients, "milk")) and not(list
contains(item.ingredients, "eggs")) then item.name
  else null
  else if Guest Profile.DietPreference = "Vegetarian" then
    if not(list contains(item.ingredients, "meat")) then item.name
  else null
  else if Guest Profile.DietPreference = "Omnivore" then
    item.name
  else
    null
```

Figure 3.8: FEEL Expression - Matches Diet

This expression ensures that meals align with the guest's dietary preferences. It applies specific filtering rules based on the selected diet type in the guest profile.

3. Matches Calories

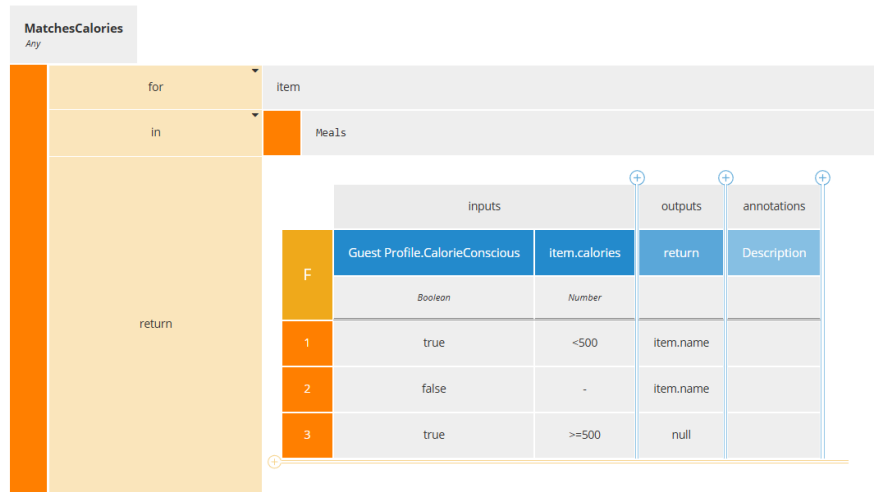


Figure 3.9: Iterator - Matches Calories

This logic is implemented as a list iterator rather than a pure FEEL expression. For every meal in the input list, it checks the `calories` field. If the guest profile indicates that the user is `CalorieConscious = true`, only meals with less than 500 calories are returned. If this preference is set to `false`, the entire input list is returned without filtering.

4. Suitable Meals

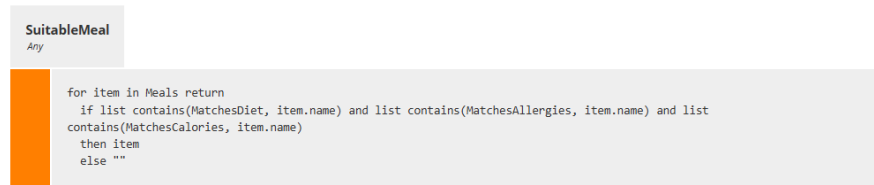


Figure 3.10: FEEL Expression - Suitable Meals

This expression aggregates the outputs from the previous filtering steps to determine which meals are fully suitable for the guest. It verifies that a meal passes all three filters: allergies, diet, and calories.

5. Sort By Course

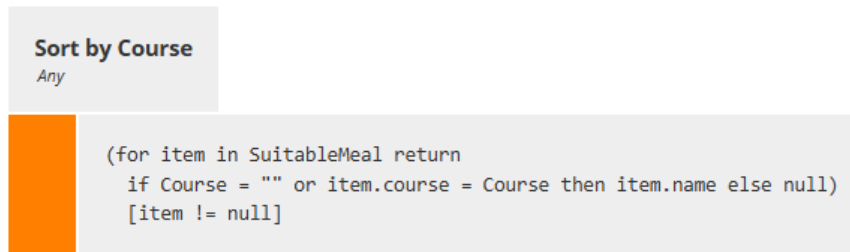


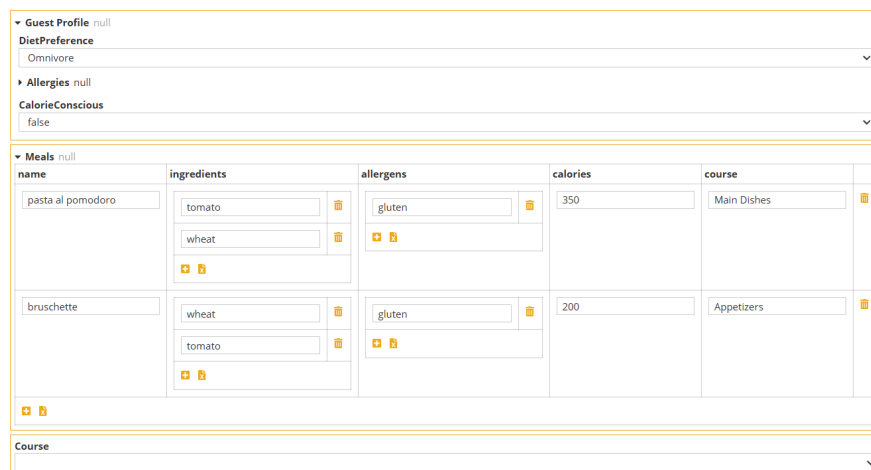
Figure 3.11: FEEL Expression - Sort by Course

This expression filters the list of suitable meals based on the selected course category (e.g., Appetizers, Pizza). If no course is selected (i.e., the input `Course` is an empty string), the full list of suitable meals is returned. Otherwise, only the meals that match the specified course are included.

3.1.4 System Execution Examples and Results

To demonstrate the functionality of the personalized menu system, we present four example executions. Each example is based on a specific user profile configuration and showcases one or more features of the system, including filtering by course, calories, dietary preference, and allergies. Input and output screenshots are provided for each case.

Example 1: No Course Specified



The screenshot shows the input form for the system. The "Guest Profile" section has "DietPreference" set to "Omnivore" and "Allergies" set to "none". The "Meals" section is a table with columns: name, ingredients, allergens, calories, and course. The "Course" field at the bottom is empty.

name	ingredients	allergens	calories	course
pasta al pomodoro	tomato wheat	gluten	350	Main Dishes
bruschette	wheat tomato	gluten	200	Appetizers

Figure 3.12: Input: No course selected, Omnivore, no allergies, no calorie restriction

In this example, the `Course` field is left empty, which results in the system not applying any filtering based on course. The user is an `Omnivore` with no allergies and no calorie concerns. This setup allows the user to see a broad selection of meals from across the entire menu, which can be particularly useful for indecisive customers who want a general overview.

Sort by Course

pasta al pomodoro

bruschette

Figure 3.13: Output: Pasta al Pomodoro, Bruschette

Example 2: Calorie-Conscious Filter for Appetizers

▼ Guest Profile null

DietPreference

Omnivore

► Allergies null

CalorieConscious

true

Figure 3.14: Input: Course = Appetizers, CalorieConscious = true, Omnivore, no allergies

This test case demonstrates the system's ability to filter meals based on calorie content. The user has selected Appetizers, is CalorieConscious, and has no dietary restrictions or allergies. The system filters out any appetizers that exceed 500 calories.

Sort by Course

Bruschette al Pomodoro

Caprese

Figure 3.15: Output: Bruschette al Pomodoro, Caprese

Example 3: Vegan Diet with Course Filtering

▼ Guest Profile null

DietPreference

Vegan

▼ Allergies null

CalorieConscious

false

Figure 3.16: Input: Course = Main Dishes, DietPreference = Vegan, no allergies, no calorie restriction

This example highlights the system's ability to recommend meals that match a specific diet preference while also considering the selected course. The guest is Vegan, with no allergies and no calorie constraints, and is interested in Main Dishes. The system excludes any meal containing meat, milk, or eggs.

Sort by Course

Risotto ai Funghi

Penne all'Arrabbiata

Figure 3.17: Output: Risotto ai Funghi, Penne all'Arrabbiata

Example 4: Vegan Diet with Gluten Allergy

▼ Guest Profile null

DietPreference

Vegan

▼ Allergies null

gluten

+

x

CalorieConscious

false

Figure 3.18: Input: Course = Main Dishes, DietPreference = Vegan, Allergy = gluten, no calorie restriction

This case builds on the previous one by adding a gluten allergy to the guest profile. The system now must filter meals that are both vegan and free of gluten allergens. The system supports multiple allergens in the allergy field, allowing for flexible and personalized filtering.

Sort by Course

Risotto ai Funghi

Figure 3.19: Output: Risotto ai Funghi

3.2 Prolog

The meal recommendation system was implemented in Prolog using a knowledge-based, modular approach to enable reasoning over dietary preferences, allergens, and nutritional constraints. The solution is designed to reflect real-world semantics, support inference, and ensure maintainability and extensibility of the knowledge base.

3.2.1 Modular and Semantic Knowledge Representation

The system represents meals using multiple specialized predicates, each encoding a distinct aspect of knowledge. This modular decomposition supports flexibility and mirrors human-understandable concepts. We will now explain one by one the predicates that compose our solution.

First, the meal predicate associates each meal with its course category, enabling course-based filtering such as appetizers, main dishes, pizzas, and desserts.

```
1 meal(bruschette_al_pomodoro, appetizer).
2 meal(caprese, appetizer).
3 meal(spaghetti_alla_carbonara, main_dish).
4 meal(risotto_ai_funghi, main_dish).
5 meal(lasagna_alla_bolognese, main_dish).
6 meal(penne_all_arrabbiata, main_dish).
7 meal(bistecca_alla_fiorentina, second_course).
8 meal(pollo_alla_cacciatora, second_course).
9 meal(melanzane_alla_parmigiana, second_course).
10 meal(margherita, pizza).
11 meal(diavola, pizza).
12 meal(prosciutto_e_funghi, pizza).
13 meal(fornarina, pizza).
14 meal(gluten_free_margherita, pizza).
15 meal(tiramisu, dessert).
16 meal(panna_cotta, dessert).
17 meal(biscotti, dessert).
```

Listing 3.1: Meal definitions associating meals with their course

Next, the meal_ingredients predicate lists all ingredients composing each meal. This information supports diet compatibility and allergen checks.

```
1 meal_ingredients(bruschette_al_pomodoro, [bread, tomato, garlic,
      olive_oil]).
2 meal_ingredients(caprese, [tomato, mozzarella, basil, olive_oil]).
3 meal_ingredients(spaghetti_alla_carbonara, [spaghetti, egg,
      pancetta, cheese]).
4 meal_ingredients(risotto_ai_funghi, [rice, mushrooms, butter,
      cheese]).
5 meal_ingredients(lasagna_alla_bolognese, [pasta, beef, cheese,
      egg]).
6 meal_ingredients(penne_all_arrabbiata, [penne, tomato, garlic,
      chili, olive_oil]).
7 meal_ingredients(bistecca_alla_fiorentina, [beef]).
8 meal_ingredients(pollo_alla_cacciatora, [chicken, tomato, olives,
      onion]).
```

```

9 meal_ingredients(melanzane_alla_parmigiana, [eggplant, cheese,
    tomato, breadcrumbs]).
10 meal_ingredients(margherita, [pizza_dough, tomato, mozzarella]).
11 meal_ingredients(diavola, [pizza_dough, tomato, mozzarella,
    salami]).
12 meal_ingredients(prosciutto_e_funghi, [pizza_dough, tomato,
    mozzarella, ham, mushrooms]).
13 meal_ingredients(fornarina, [pizza_dough, olive_oil]).
14 meal_ingredients(gluten_free_margherita, [gluten_free_dough,
    tomato, mozzarella]).
15 meal_ingredients(tiramisu, [mascarpone, egg, coffee, ladyfingers,
    cocoa]).
16 meal_ingredients(panna_cotta, [cream, milk, sugar]).
17 meal_ingredients(biscotti, [flour, egg, butter, sugar]).

```

Listing 3.2: Ingredients composing each meal

Than the `meal_calories` predicate records the calorie content per meal, which supports filtering for calorie-conscious guests.

```

1 meal_calories(bruschette_al_pomodoro, 180).
2 meal_calories(caprese, 250).
3 meal_calories(spaghetti_alla_carbonara, 565).
4 meal_calories(risotto_ai_funghi, 450).
5 meal_calories(lasagna_alla_bolognese, 750).
6 meal_calories(penne_all_arrabbiata, 325).
7 meal_calories(bistecca_alla_fiorentina, 655).
8 meal_calories(pollo_alla_cacciatora, 570).
9 meal_calories(melanzane_alla_parmigiana, 550).
10 meal_calories(margherita, 600).
11 meal_calories(diavola, 650).
12 meal_calories(prosciutto_e_funghi, 700).
13 meal_calories(fornarina, 250).
14 meal_calories(gluten_free_margherita, 850).
15 meal_calories(tiramisu, 450).
16 meal_calories(panna_cotta, 250).
17 meal_calories(biscotti, 125).

```

Listing 3.3: Calories associated with each meal

The `meal_allergens` predicate enumerates allergens present in each meal, enabling safety checks against guest allergies.

```

1 meal_allergens(bruschette_al_pomodoro, [gluten]).
2 meal_allergens(caprese, [milk]).
3 meal_allergens(spaghetti_alla_carbonara, [gluten, milk, egg]).
4 meal_allergens(risotto_ai_funghi, [milk]).
5 meal_allergens(lasagna_alla_bolognese, [gluten, milk, egg]).
6 meal_allergens(penne_all_arrabbiata, [gluten]).
7 meal_allergens(bistecca_alla_fiorentina, []).
8 meal_allergens(pollo_alla_cacciatora, []).
9 meal_allergens(melanzane_alla_parmigiana, [gluten, milk]).
10 meal_allergens(margherita, [gluten, milk]).
11 meal_allergens(diavola, [gluten, milk]).
12 meal_allergens(prosciutto_e_funghi, [gluten, milk]).

```



```

13 meal_allergens(fornarina, [gluten]).
14 meal_allergens(gluten_free_margherita, [milk]).
15 meal_allergens(tiramisu, [gluten, milk, egg]).
16 meal_allergens(panna_cotta, [milk]).
17 meal_allergens(biscotti, [gluten, milk, egg]).

```

Listing 3.4: Allergens associated with each meal

Finally, the `ingredient_type` predicate categorizes ingredients by type (e.g., meat, dairy, vegetable), this is useful for applying diet restrictions semantically.

```

1 ingredient_type(bread, cereal).
2 ingredient_type(pizza_dough, cereal).
3 ingredient_type(gluten_free_dough, cereal).
4 ingredient_type(tomato, vegetable).
5 ingredient_type(garlic, vegetable).
6 ingredient_type(olive_oil, plant_fat).
7 ingredient_type(basil, vegetable).
8 ingredient_type(mozzarella, dairy).
9 ingredient_type(cheese, dairy).
10 ingredient_type(pancetta, meat).
11 ingredient_type(salami, meat).
12 ingredient_type(ham, meat).
13 ingredient_type(beef, meat).
14 ingredient_type(chicken, meat).
15 ingredient_type(mushrooms, vegetable).
16 ingredient_type(eggplant, vegetable).
17 ingredient_type(rice, cereal).
18 ingredient_type(pasta, cereal).
19 ingredient_type(penne, cereal).
20 ingredient_type(spaghetti, cereal).
21 ingredient_type(egg, animal_product).
22 ingredient_type(butter, dairy).
23 ingredient_type(cream, dairy).
24 ingredient_type(milk, dairy).
25 ingredient_type(sugar, sweetener).
26 ingredient_type(cocoa, plant_product).
27 ingredient_type(coffee, plant_product).
28 ingredient_type(ladyfingers, cereal).
29 ingredient_type(flour, cereal).
30 ingredient_type(onion, vegetable).
31 ingredient_type(olives, vegetable).
32 ingredient_type(mascarpone, dairy).

```

Listing 3.5: Ingredient type classifications

Each of the predicate contributes to a clear, granular knowledge base that enables reasoning over individual properties like ingredients, calorie content, and allergens.

3.2.2 Ontological Reasoning and Diet Constraints

The system was structured in a way to resemble an ontology for the ingredient types (e.g., meat, dairy, vegetable) and expresses dietary requirements through constraints on these types. This supports logical inference instead of hardcoding incompatibilities and allows for a manageable and maintainable system. the predicate below, shows the association between certain kind of ingredient and certain diets.

```
1 diet_restriction(vegetarian, meat).
2 diet_restriction(vegan, meat).
3 diet_restriction(vegan, dairy).
4 diet_restriction(vegan, animal_product).
```

Listing 3.6: Dietary restrictions by ingredient type

For most diets (e.g., vegan, vegetarian), compatibility is enforced by ensuring that no disallowed ingredient types are present in a meal. The **carnivore** diet however, behaves differently: it is defined by a requirement rather than a restriction. Meals suitable for carnivores must include at least one meat-based ingredient. This distinction is explicitly encoded in the logic.

```
1 % Diet compatibility rules
2 meal_satisfies_diet(Meal, carnivore) :-
3     meal_ingredients(Meal, Ingredients),
4     member(Ing, Ingredients),
5     ingredient_type(Ing, meat).
6
7 meal_satisfies_diet(Meal, Diet) :-
8     Diet \= carnivore,
9     meal_ingredients(Meal, Ingredients),
10    \+ (member(Ing, Ingredients),
11        ingredient_type(Ing, Type),
12        diet_restriction(Diet, Type)).
```

Listing 3.7: Diet compatibility inference rules

To further filter recommendations, after checking the diet, the system checks for allergen conflicts and calorie limits. These rules ensure that a recommended meal is not only diet-compatible but also safe and appropriate for the user's health constraints.

```
1 % Allergy safety rule
2 meal_safe_for_allergies(Meal, Allergies) :-
3     meal_allergens(Meal, MealAllergens),
4     \+ (member(Allergen, Allergies),
5         member(Allergen, MealAllergens)).
6
7 % Calorie constraint rule
8 meal_within_calories(Meal, MaxCalories) :-
9     meal_calories(Meal, Cals),
10    Cals <= MaxCalories.
```

Listing 3.8: Allergy safety and calorie constraints

Main Recommendation Logic

The core recommendation rule integrates all filtering layers—course type, dietary compatibility, allergy safety, and calorie threshold—into a unified predicate called `recommend_meal` that takes in input the Course, the Diet, a list of allergies, a boolean to specify if the user is calorie conscious and the maximum amount of calories that a calorie conscious customer is willing to eat. Below is a listing showing the `recommend_meal` predicate.

```
1 recommend_meal(Course, Diet, Allergies, false, _, Meal) :-  
2     meal(Meal, Course),  
3     meal_satisfies_diet(Meal, Diet),  
4     meal_safe_for_allergies(Meal, Allergies).  
5  
6 recommend_meal(Course, Diet, Allergies, true, MaxCals, Meal) :-  
7     meal(Meal, Course),  
8     meal_satisfies_diet(Meal, Diet),  
9     meal_safe_for_allergies(Meal, Allergies),  
10    meal_within_calories(Meal, MaxCals).
```

Listing 3.9: Meal recommendation rules

3.2.3 Prolog-Based Execution Results

In this section, we present the results obtained by executing our knowledge-based recommendation system implemented in Prolog. Each example case simulates a different guest profile and uses the same knowledge base to recommend suitable meals according to course, dietary preferences, allergies, and calorie restrictions.

Example 1: Appetizer, Omnivore, No Allergy, No Calorie Restrictions

This example checks if the system correctly filters meals based on the selected course. The guest is an Omnivore, with no allergies and no calorie restrictions, and has selected Appetizer as the desired course.



The screenshot shows a Prolog query window with the query `recommend_meal(appetizer, omnivore, [], false, _, Meal).` and a table of results. The table has a header row with the title "Meal" and a column for the meal name and a column for a numerical value.

Meal	
bruschette_al_pomodoro	1
caprese	2

Figure 3.20: Example 1 Output: Appetizers for Omnivore

Example 2: Main Dishes, Vegetarian, No Allergy, No Calorie Restriction

This example demonstrates how dietary preferences are enforced. The guest is Vegetarian, and has selected Main Dishes. The system filters out meals that contain meat but accepts dairy and egg-based dishes.

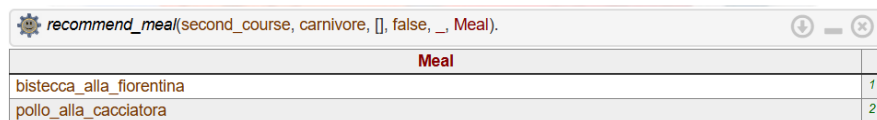


Meal	
risotto_ai_funghi	1
penne_all_arrabbiata	2

Figure 3.21: Example 2 Output: Vegetarian Main Dishes

Example 3: Second Course, Carnivore, No Allergy, No Calorie Restriction

This example illustrates how the Carnivore preference works differently compared to other diets. Instead of excluding ingredients, it explicitly looks for meals that contain meat. The course filter is also active, targeting Second Course options.

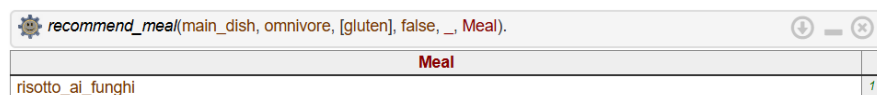


Meal	
bistecca_alla_florentina	1
pollo_alla_cacciatora	2

Figure 3.22: Example 3 Output: Meat-Based Second Courses for Carnivore

Example 4: Main Dishes, Omnivore, Allergic to Gluten, No Calorie Restriction

This example focuses on the allergy filtering logic. The guest is an Omnivore with a gluten allergy. The system should exclude any meals containing gluten while maintaining compatibility with the selected course.

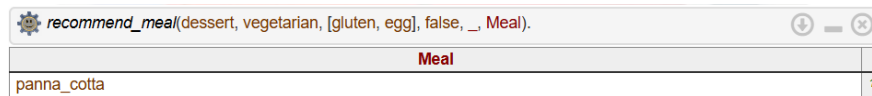


Meal	
risotto_ai_funghi	1

Figure 3.23: Example 4 Output: Gluten-Free Main Dishes

Example 5: Dessert, Vegetarian, Allergic to Gluten and Egg, No Calorie Restriction

This example demonstrates how the system handles multiple allergies. The guest is Vegetarian, allergic to both gluten and egg, and interested in Dessert. The system must exclude any desserts containing either of the allergens.



⚙️ <code>recommend_meal(dessert, vegetarian, [gluten, egg], false, _, Meal).</code>		⬆️ ⬇️ ⬵
Meal		
panna_cotta		1

Figure 3.24: Example 5 Output: Gluten- and Egg-Free Desserts

Example 6: Dessert, Vegetarian, Allergic to Nuts, Calorie-Conscious (Under 300 Calories)

This final example combines allergy filtering with calorie restriction. The guest is Vegetarian, allergic to nuts, and calorie-conscious, preferring meals under 300 calories.



⚙️ <code>recommend_meal(dessert, vegetarian, [nuts], true, 300, Meal).</code>		⬆️ ⬇️ ⬵
Meal		
panna_cotta		1
biscotti		2

Figure 3.25: Example 6 Output: Low-Calorie Nut-Free Vegetarian Desserts

3.3 Ontology

3.3.1 Ontology structure

The ontology developed in this project represents one of three distinct knowledge representation approaches, with DMN and Prolog, used to model and reason about meal personalization scenarios. Its purpose is not to interoperate directly with the other two paradigms, but rather to offer a semantic, logic-based framework for representing domain knowledge.

This allows a structured comparison between rule-based models (Prolog), decision logic (DMN), and formal ontologies (OWL) in terms of expressiveness, modularity, and reasoning capabilities. In this context, an ontology is a structured, machine-interpretable model of concepts, properties, and relationships within a domain. Developed with Protégé and expressed in OWL, the ontology in this project enables personalized meal recommendations tailored to user preferences, dietary restrictions, and allergies, and can be further extended to support additional use cases or requirements.

The core of the ontology is centered around the customization of meal choices for guests, represented through four main classes: Diet, Guest, Ingredient, and Meal. Each of these classes captures a distinct conceptual aspect of the food recommendation system. The figure 3.26 represents the main classes of our ontology and the structured that we decided to have.

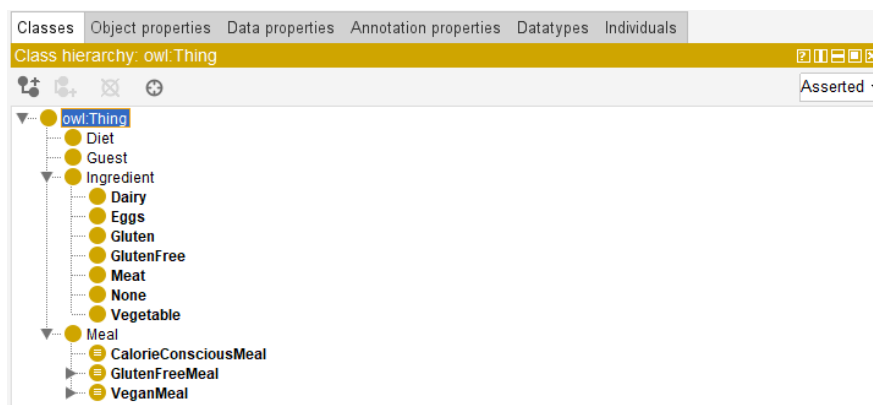


Figure 3.26: Structure of the classes

The Guest class models the user and links to their dietary needs or restrictions, while Meal represents the available dishes that can be filtered or selected according to those needs. Diet and Ingredient serve as descriptors of what a guest prefers or can consume.

The Ingredient class is further refined into a hierarchy of more specific subclasses, such as Dairy, Eggs, Gluten, GlutenFree, Meat, None, and Vegetable. These subclasses help identify both the composition of meals and the allergens or exclusions associated with a guest. The decision to refine the Ingredient class into a hierarchy of specific subclasses, such as Dairy, Eggs, Gluten, GlutenFree, Meat, None, and Vegetable, was driven by the need for semantic clarity and reasoning precision. In the context of personalized meal recommendations, ingredients serve a dual function: they define the composition of meals and represent potential allergens or dietary restrictions. Organizing them into

a structured class hierarchy enhances both maintainability and extensibility, enabling the ontology to easily accommodate new dietary categories or ingredients over time.

The figure 3.27 illustrates one such subclass, Dairy, which contains only instances of ingredients derived from milk or similar sources. This categorization allows for a more detailed and semantically rich representation of ingredients and allergies. With this approach, adding a new meal becomes more straightforward, if the ingredients are correctly classified within the ontology, inference mechanisms can automatically determine dietary compatibility without requiring additional manual rules or updates.

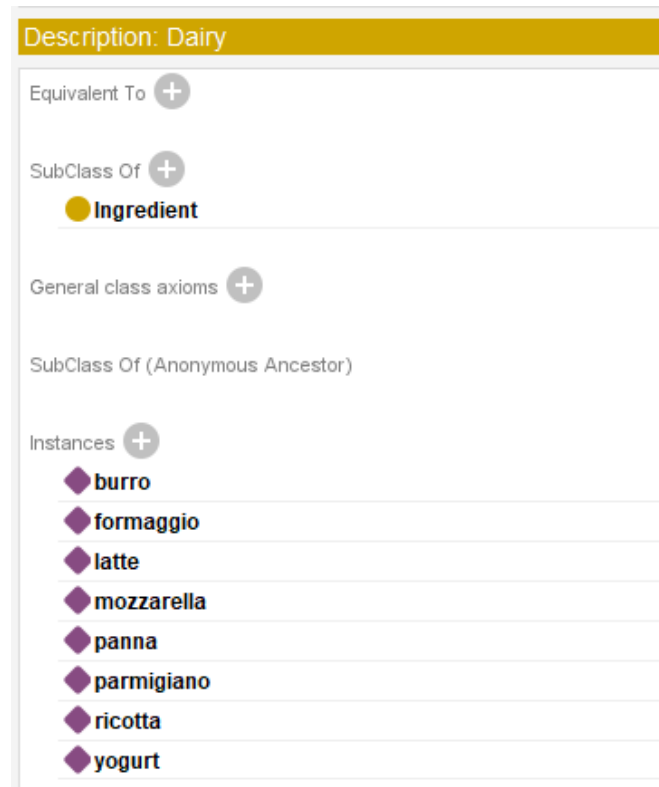


Figure 3.27: Structure of the classes

In parallel, the Meal class is specialized into a number of subclasses including CalorieConsciousMeal, GlutenFreeMeal, and VeganMeal, with further refinement such as GFVegMeal and GFCalMeal as subclasses of GlutenFreeMeal, and VegCalMeal as a subclass of VeganMeal.

This hierarchical structure was chosen to capture overlapping dietary properties in a modular and logically consistent way. Rather than assigning multiple unrelated attributes to each meal, subclassing allows us to semantically group meals based on shared nutritional or dietary constraints. For example, a GFCalMeal simultaneously satisfies the conditions for being gluten-free and calorie-conscious, and its membership can be inferred automatically based on defined property restrictions (e.g., `notContainsGluten = true` and `hasCalories < 500`) as we can see in Figure 3.28. Similarly, GlutenFreeMeal is characterized by a boolean data property `notContainsGluten` set to true, and VeganMeal by `isVegan` set to true. These definitions enable automatic classification and reasoning, allowing the ontology to infer membership based on asserted facts about a

meal.



Figure 3.28: CalorieConsciousMeal definition

The relationships among the concepts are modeled through object properties. A `Guest` may have one or more allergies represented via the `hasAllergy` property, which ranges over ingredient types such as `Dairy`, `Gluten`, or `Eggs`. The `hasDiet` property links a `Guest` to a specific `Diet`, while the `hasIngredient` property associates a `Meal` with its ingredients.

Most critically for personalization, the `hasSuitableMeal` property connects a `Guest` with recommended `Meal` instances based on their profile. In this figure 3.29 we can see an example of how to configure domain and ranges for Object Properties.

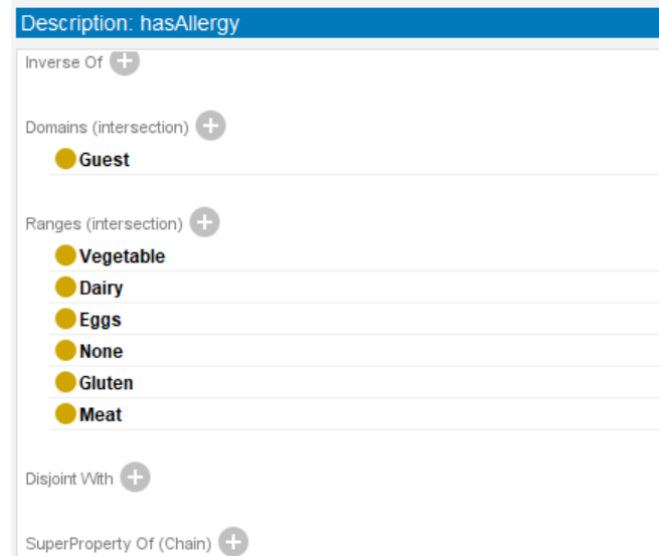


Figure 3.29: Object Property: Guest hasDiet Diet

The ontology uses data properties to represent literal values that support reasoning tasks such as classification, filtering, and rule-based inference. Properties like `hasCalories`, which indicates the energy content of a meal as an integer, and `hasCourse`, which describes the type of dish as a string, allow the ontology to capture specific characteristics of individual meals. Other properties such as `isCalorieConscious`, `isVegan`, and `notContainsGluten` are modeled as boolean values that describe key dietary traits. These are defined as data properties rather than object properties because they link individuals to concrete values rather than to other individuals or classes. For example, a meal may have a calorie value of 450 or a boolean value of `true` for being vegan, which are best represented through direct literal values. This approach maintains clarity in

the ontology's structure and allows reasoners to efficiently apply logical rules and class conditions based on numerical thresholds or binary attributes. In this figure 3.30, we can see an example for the configuration of isCalorieConscious.

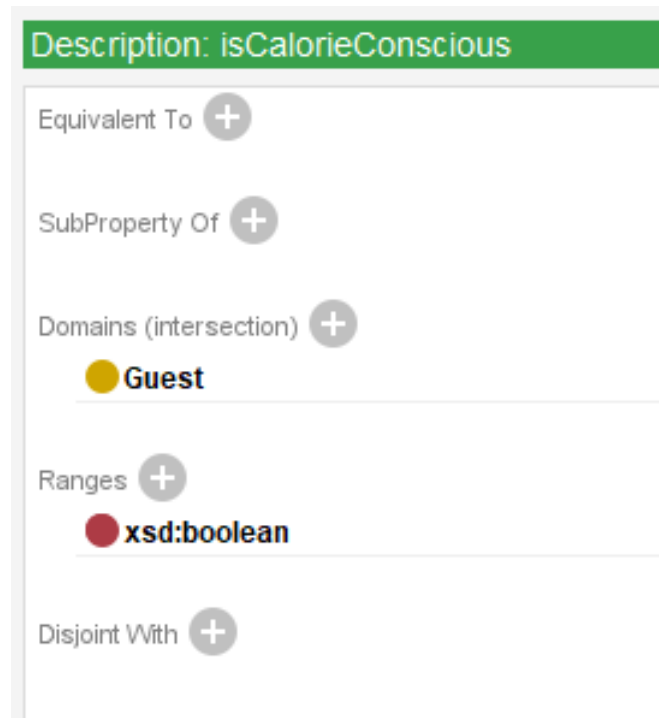


Figure 3.30: Data Property: Guest isCalorieConscious boolean

3.3.2 SWRL Rules

To capture complex personalization logic that cannot be easily expressed through standard class definitions, the ontology employs SWRL rules. For example, the GFOnlyRule specifies that if a guest is allergic to gluten, is not calorie-conscious, and follows an omnivorous diet, then any gluten-free meal should be considered suitable for that guest. Similar rules exist for every combination of gluten allergy status, calorie-consciousness, and dietary preference, allowing the system to comprehensively handle nuanced guest profiles.

Using SWRL rules in this way provides greater expressivity and flexibility by enabling conditional reasoning based on multiple factors simultaneously. This approach allows the ontology to infer suitable meal recommendations by integrating explicit guest preferences, dietary restrictions, and allergies within a coherent logical framework. On the positive side, rules make it straightforward to encode complex decision criteria without overloading class hierarchies or complicating property definitions. They also make the reasoning process more transparent and easier to modify as new conditions or combinations arise.

However, this approach comes with trade-offs. The use of SWRL rules can increase reasoning complexity and impact performance, especially as the number of rules and data

instances grows. Additionally, rule management can become unmanageable without proper organization, and ensuring completeness and consistency across all combinations requires careful design. In the figures 3.31 and 3.32 we can see two different rules, one for people who have a gluten allergy as a dietary restriction only and the other one for the most restrictive diet. In the last figure 3.33 we can see of the implemented rule for every combination of Vegan, Gluten Free and Calorie conscious.

Figure 3.31: Most Restrictive Rule: celiac person, vegan and calorie conscious

Figure 3.32: Rule for a celiac person

	Name	Rule	Comment
✓	GFOOnlyRule	GlutenFreeMeal(?m) ^ hasAllergy(?g, gluten) ^ isCalorieConscious(?g, false) ^ hasDiet(?g, Omnivore) ^ Guest(?g) -> hasSuitableMeal(?g, ?m)	
✓	CalorieConsciousRule	hasAllergy(?g, gluten) ^ hasDiet(?g, Omnivore) ^ isCalorieConscious(?g, true) ^ hasDiet(?g, Omnivore) ^ Guest(?g) -> hasSuitableMeal(?g, ?m)	
✓	OnlyCalorieConsciousRule	hasAllergy(?g, gluten) ^ hasDiet(?g, Omnivore) ^ isCalorieConscious(?g, true) ^ hasDiet(?g, Omnivore) ^ Guest(?g) -> hasSuitableMeal(?g, ?m)	
✓	RestrictedDiet	isCalorieConscious(?g, true) ^ hasDiet(?g, Vegan) ^ hasAllergy(?g, gluten) ^ GFCalVeg(?m) ^ Guest(?g) -> hasSuitableMeal(?g, ?m)	
✓	VegCalRule	hasAllergy(?g, gluten) ^ isCalorieConscious(?g, true) ^ hasDiet(?g, Vegan) ^ hasDiet(?g, Vegan) ^ Guest(?g) -> hasSuitableMeal(?g, ?m)	
✓	VegGFOOnlyRule	hasAllergy(?g, gluten) ^ hasDiet(?g, Vegan) ^ hasDiet(?g, Vegan) ^ isCalorieConscious(?g, false) ^ Guest(?g) -> hasSuitableMeal(?g, ?m)	

Figure 3.33: List of Rules

3.3.3 Individuals

The ontology was populated with individual instances representing guests and meals to simulate realistic scenarios for personalized meal recommendations. By assigning specific attributes such as allergies, dietary preferences and meal compositions to these instances, the system was able to apply the defined SWRL rules and ontology axioms effectively. Through iterative testing, the reasoning engine successfully inferred suitable meals for each guest, filtering out options that did not meet their dietary constraints or preferences. This validation process confirmed that the ontology, combined with rule-based reasoning, accurately models the personalization logic and delivers appropriate meal recommendations aligned with each guest's profile.

After designing the SWRL rules and SHACL validation, we activated the reasoner provided by Protégé. Several parts of the ontology rely on inference. For example, the types of meals are automatically inferred based on class expressions that categorize dishes as vegan, gluten-free, and so on. The most important outcomes, however, are the results produced by the SWRL rules, which combine multiple constraints to determine suitable meal recommendations.

As shown in the pictures 3.34 3.35 3.36 below we have the results after the reasoning and Protégé successfully inferred the type of meal and the meal suggestions.

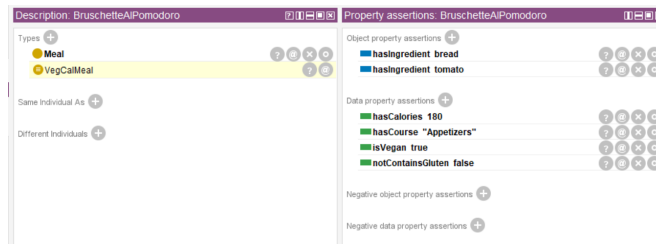


Figure 3.34: Inferred Meal

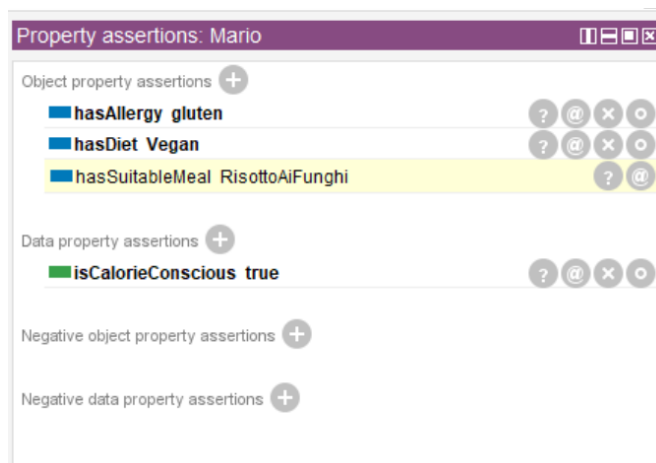


Figure 3.35: Inferred rule for one guest

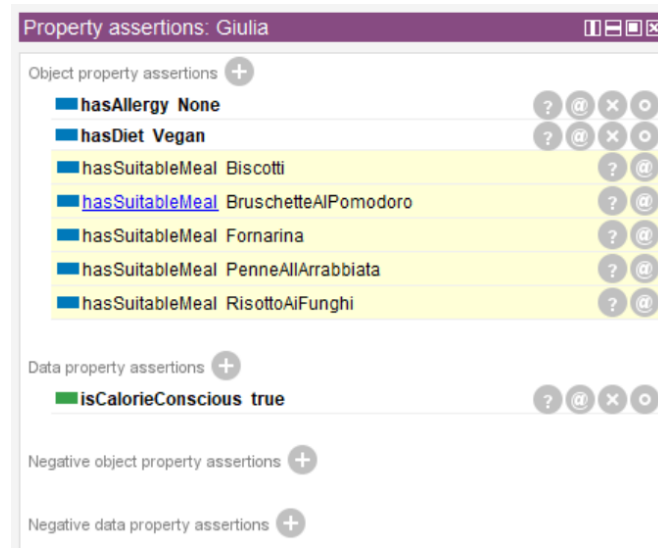


Figure 3.36: Inferred rule for one guest

3.3.4 SHACL Shapes and SPARQL Query

To ensure data quality and consistency within the ontology, SHACL (Shapes Constraint Language) validations were implemented. In the figure, you can the one develop for Guest. These validations serve to enforce domain-specific constraints, such as mandatory properties, value ranges, and logical conditions, directly on the data level. By applying SHACL shapes, the system can automatically detect and report violations, such as missing allergy information for a guest or incorrect calorie values for a meal—before reasoning is performed. This pre-validation step helps maintain the integrity of the data, reduces errors, and improves the reliability of subsequent inference processes.

In addition to SHACL validation, the project employs queries to retrieve and analyze specific information from the ontology data. Queries allow for more flexible data inspection and scenario testing compared to SHACL, which primarily focuses on enforcing constraints and validating the structural and semantic correctness of the data.

```

1 @prefix rdf:    <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix sh:     <http://www.w3.org/ns/shacl#> .
3 @prefix xsd:    <http://www.w3.org/2001/XMLSchema#> .
4 @prefix rdfs:   <http://www.w3.org/2000/01/rdf-schema#> .
5 @prefix :       <http://www.semanticweb.org/ontologies/2025/personalized-menu/>
6 @prefix owl:  <http://www.w3.org/2002/07/owl#> .
7
8
9 :MealShape
10   a sh:NodeShape ;
11   sh:targetClass :Meal ;
12
13   sh:property [
14     sh:path :hasIngredient ;
15     sh:minCount 1 ;
16     sh:class :Ingredient ;
17   ] ;
18

```

```

19     sh:property [
20         sh:path :isVegan ;
21         sh:datatype xsd:boolean ;
22         sh:minCount 1 ;
23     ] ;
24
25     sh:property [
26         sh:path :hasCalories ;
27         sh:datatype xsd:integer ;
28         sh:minCount 1 ;
29     ] ;
30
31     sh:property [
32         sh:path :hasCourse ;
33         sh:datatype xsd:string ;
34         sh:minCount 1 ;
35     ] ;
36
37     sh:property [
38         sh:path :notContainsGluten ;
39         sh:datatype xsd:boolean ;
40         sh:minCount 1 ;
41     ] .

```

Listing 3.10: SHACL validation for Meal Shape

```

1 @prefix rdf:    <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix sh:     <http://www.w3.org/ns/shacl#> .
3 @prefix xsd:    <http://www.w3.org/2001/XMLSchema#> .
4 @prefix rdfs:   <http://www.w3.org/2000/01/rdf-schema#> .
5 @prefix :      <http://www.semanticweb.org/ontologies/2025/personalized-menu/>
6 @prefix owl:  <http://www.w3.org/2002/07/owl#> .
7
8
9 :GuestShape
10     a sh:NodeShape ;
11     sh:targetClass :Guest ;
12
13     sh:property [
14         sh:path :hasAllergy ;
15         sh:minCount 1 ;
16     ] ;
17
18     sh:property [
19         sh:path :hasDiet ;
20         sh:minCount 1 ;
21     ] ;
22
23     sh:property [
24         sh:path :isCalorieConscious ;
25         sh:datatype xsd:boolean ;
26         sh:minCount 1 ;
27     ] .

```

Listing 3.11: SHACL validation for Guest Shape

Queries were executed both using Protégé and AOAME, a specialized platform offering advanced capabilities for querying and managing ontologies. While both tools support SPARQL queries, they differ in terms of user interface, performance, and integrated features for result analysis. Protégé provides a general, user-friendly environment, whereas AOAME emphasizes advanced and customizable functions for complex analysis.

In the figure 3.37 there is a simple query run in Protégé to find all the meals under 400 calories just to check if everything is working but for more complex queries AOAME provides more functionalities to produce complex queries.

The screenshot shows a SPARQL query window in Protégé. The query is as follows:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX : <http://www.semanticweb.org/angel/ontologies/2025/5/personalized-menu#>

SELECT ?meal ?calories
WHERE {
  ?meal rdf:type :Meal.
  ?meal isVegan true.
  ?meal hasCalories ?calories .
  FILTER (?calories < 400)
}
ORDER BY ?calories

```

Below the query, a table displays the results. The table has two columns: 'meal' and 'calories'.

meal	calories
Biscotti	"125"^^<http://www.w3.org/2001/XMLSchema#integer>
BruschettaAlPomodoro	"180"^^<http://www.w3.org/2001/XMLSchema#integer>
Fornarina	"250"^^<http://www.w3.org/2001/XMLSchema#integer>
PenneAllArabbiata	"325"^^<http://www.w3.org/2001/XMLSchema#integer>

Figure 3.37: A simple query in Protégé

Using both tools for querying leverages their respective strengths and ensures thorough and versatile validation of the ontology. The differences between these two environments will be discussed in greater detail in later sections of the document.

4. Agile and Ontology-based Meta-modelling

In this chapter, we analyze the integration of agile and ontology-based meta-modelling with BPMN 2.0, extending the standard to incorporate domain-specific semantics tailored for meal recommendation processes. Building on the ontology developed in the previous chapter, we enrich it with additional classes and properties, aligning it with BPMN 2.0 tasks to represent the meal recommendation system.

The extended ontology is deployed on a Jena Fuseki triple store, which we use to execute SPARQL queries that drive the personalized meal recommendations. In the following sections, we introduce the Agile Ontology-based Adaptive Meta-Modelling Environment (AOAME), present our extended BPMN 2.0 model, and walk through example queries to demonstrate the system’s capabilities and outputs.

4.1 AOAME

Agile Ontology-based Adaptive Meta-Modelling Environment (AOAME) is a tool that combines agile principles with ontology-driven modeling to support the rapid development and adaptation of domain-specific modeling tools. AOAME promotes flexibility, modularity, and semantic clarity in model-driven development by enabling the creation of meta-models that can adapt dynamically to evolving requirements. In the context of this project, we used AOAME for Task 2, to extend BPMN 2.0 with semantic annotations and domain-specific knowledge from an ontology representing food, meals, dietary constraints, and customer preferences. This allows for a more intelligent and personalized process modeling environment, capable of integrating reasoning and recommendation logic directly into the process models.

4.2 BPMN 2.0

Business Process Model and Notation (BPMN) 2.0 is a standardized graphical notation developed by the Object Management Group (OMG) for specifying business processes in a workflow. It provides a rich set of diagram elements, such as events, tasks, gateways, and flows, to capture both the procedural logic and business semantics of processes. BPMN 2.0 aims to bridge the gap between technical developers and business stakeholders by offering a modeling language that is expressive yet intuitive. In this project, BPMN 2.0 is extended with domain-specific semantics using ontology-based annotations, particularly through the creation of new task types related to meal recommendation. This extension facilitates the execution of context-aware processes

that reason over structured knowledge bases.

4.2.1 Our BPMN Model

The BPMN model developed for this project represents the interaction between the customer and the restaurant’s meal recommendation system. The process is modeled using a single **pool** that is divided into two **lanes**: one for the *Customer* and one for the *System*. This separation clearly delineates the responsibilities and interactions of each actor within the business process.

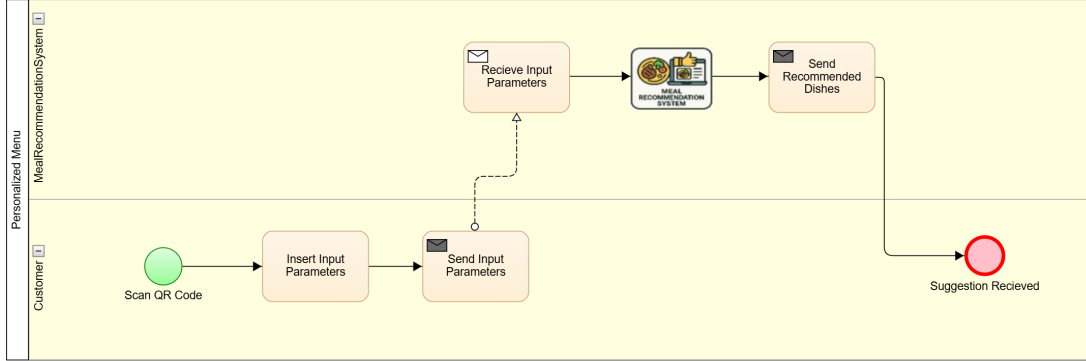


Figure 4.1: BPMN 2.0 Model for the Meal Recommendation Process

The process begins when the customer scans a QR code at the restaurant to access the digital menu. This is represented as a *Start Event* in the Customer lane. Following this, the customer performs a task labeled **Insert Input Parameters**, where they input key information such as allergies, dietary preferences (e.g., Vegan, Celiac, Carnivore), preferred course (e.g., appetizer, main dish), and their maximum calorie intake.

Once the input parameters are confirmed, a task named **Send Input Parameters** sends this information to the system via a *Message Flow*. On the System lane, a corresponding task **Receive Input Parameters** captures the input data.

Next, the core of the system is represented by a custom task labeled **Meal Recommendation System**. This task is an ontology-extended BPMN task that leverages the knowledge graph and SPARQL queries to determine which meals best suit the customer’s profile. This custom task is modeled as an extension of the BPMN 2.0 Task element and is tied to the underlying semantic logic implemented in Jena Fuseki.

After the meal recommendation has been computed, the system executes the task **Send Recommended Dishes**, which sends the result back to the Customer lane. The final step in the process is a task labeled **Recommendation Received**, which marks the end of the process from the customer’s perspective.

In particular, the **Meal Recommendation System** task is based on a custom class called `RecommendMeals`, which extends the standard BPMN Task class. This class is enriched with specific data properties that represent the user input collected during the process. These properties are directly mapped to the semantic model and used to trigger SPARQL queries over the ontology.

- `hasCoursePreference` (string)
- `hasAllergy` (string)
- `hasDiet` (string)
- `isCalorieConscious` (boolean)
- `maxCalorieIntake` (int)

ID: RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10
Instantiation Type: Instance

Relation	Value	Actions
hasCoursePreference	Dessert	Remove
hasAllergy	nuts	Remove
hasDiet	Vegan	Remove
isCalorieConscious	true	Remove
maxCalorieIntake	600	Remove

▼ Add Relation

Save Close

Figure 4.2: Extended Task Class: `RecommendMeals` and its Data Properties

4.3 Jena Fuseki

Jena Fuseki is a SPARQL server and Linked Data platform that allows users to query RDF datasets using the SPARQL query language. It is part of the Apache Jena framework and supports storage, inference, and querying of RDF graphs. In this project, Jena Fuseki serves as the backend triple store for the ontology. It enables the execution of SPARQL queries that retrieve personalized meal recommendations based on customer preferences, dietary restrictions, and process context. Fuseki allows seamless integration of the knowledge graph with the extended BPMN 2.0 model, making ontology-driven business processes executable and intelligent.

4.3.1 Our Jena Fuseki Query

On the Jena Fuseki server, we defined a SPARQL query (see Figure 4.3) that retrieves a personalized meal recommendation from the ontology developed in Chapter 3. This query is based on the model defined in AOAME, as described in the previous section, and leverages the extended `RecommendMeals` class and its associated properties to dynamically extract meals tailored to a user's preferences.

Compared to the version used earlier in the ontology section, this new query introduces several improvements:

- The retrieval of user preferences is performed directly from the extended ontology class, rather than hardcoding the values in the query.
- Additional prefixes are introduced for clarity and broader semantic support.
- More refined filters are added to handle complex preferences, including dietary types such as **Carnivore**, which was not fully implemented in the original version.
- Users can now specify a custom `maxCalorieIntake` instead of relying on a fixed default.

These enhancements allow the query to more effectively tailor recommendations according to the user’s allergies, dietary restrictions, and caloric constraints.

The SPARQL query shown in the figure below is designed to retrieve a list of meal names that match a user’s personalized dietary preferences. It begins by selecting the extended `RecommendMeals` individual and extracting its properties, including `hasCoursepreference`, `hasAllergy`, `hasDiet`, `isCalorieConscious`, and `maxCalorieIntake`. These parameters represent the user’s input and guide the filtering process. The query then identifies candidate meals that match the desired course and evaluates optional properties such as ingredients, calorie content, whether the meal is vegan, and whether it is gluten-free. The allergy filter checks if a specific allergy is declared and, for each known allergen (e.g., milk, nuts, soybeans), ensures that the corresponding ingredient is not present in the meal. The calorie filter applies only if the user has enabled calorie-conscious recommendations, filtering out meals exceeding the specified limit. Diet-related constraints are handled through additional filters: vegan meals must have the `isVegan` property set to true, gluten-free meals (for celiac users) must have `notContainsGluten` set to true, and carnivore users are only recommended meals that explicitly include `per:meat` as an ingredient. Finally, the result returns only the local name of the meal URI using the `STRFTER` function for readability.

```

1 PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
2 PREFIX mod: <http://fhnw.ch/modelingEnvironment/ModelOntology#>
3 PREFIX lo: <http://fhnw.ch/modelingEnvironment/LanguageOntology#>
4 PREFIX per: <http://www.semanticweb.org/angel/ontologies/2025/5/personalized-menu#>
5 PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
6
7 SELECT DISTINCT (STRAFTER(STR(?meal), "#") AS ?mealName)
8 WHERE {
9   mod:RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10 lo:hasCoursepreference ?course .
10  mod:RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10 lo:hasAllergy ?allergy .
11  mod:RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10 lo:hasDiet ?diet .
12  mod:RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10 lo:isCalorieConscious ?isCalorieConscious .
13  mod:RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10 lo:maxCalorieIntake ?maxCalories .
14
15  ?meal rdf:type per:Meal ;
16         per:hasCourse ?course .
17
18  OPTIONAL { ?meal per:hasIngredient ?ingredient . }
19  OPTIONAL { ?meal per:hasCalories ?mealCalories . }
20  OPTIONAL { ?meal per:isVegan ?isVegan . }
21  OPTIONAL { ?meal per:notContainsGluten ?notContainsGluten . }
22
23  FILTER (
24    !BOUND(?allergy) ||
25    (LCASE(?allergy) != "celery" || !EXISTS { ?meal per:hasIngredient per:celery }) &&
26    (LCASE(?allergy) != "cereals" || !EXISTS { ?meal per:hasIngredient per:cereals }) &&
27    (LCASE(?allergy) != "crustaceans" || !EXISTS { ?meal per:hasIngredient per:crustaceans }) &&
28    (LCASE(?allergy) != "eggs" || !EXISTS { ?meal per:hasIngredient per:eggs }) &&
29    (LCASE(?allergy) != "fish" || !EXISTS { ?meal per:hasIngredient per:fish }) &&
30    (LCASE(?allergy) != "lupin" || !EXISTS { ?meal per:hasIngredient per:lupin }) &&
31    (LCASE(?allergy) != "milk" || !EXISTS { ?meal per:hasIngredient per:milk }) &&
32    (LCASE(?allergy) != "molluscs" || !EXISTS { ?meal per:hasIngredient per:molluscs }) &&
33    (LCASE(?allergy) != "mustard" || !EXISTS { ?meal per:hasIngredient per:mustard }) &&
34    (LCASE(?allergy) != "peanuts" || !EXISTS { ?meal per:hasIngredient per:peanuts }) &&
35    (LCASE(?allergy) != "sesame" || !EXISTS { ?meal per:hasIngredient per:sesame }) &&
36    (LCASE(?allergy) != "soybeans" || !EXISTS { ?meal per:hasIngredient per:soybeans }) &&
37    (LCASE(?allergy) != "sulphites" || !EXISTS { ?meal per:hasIngredient per:sulphites }) &&
38    (LCASE(?allergy) != "nuts" || !EXISTS { ?meal per:hasIngredient per:nuts })
39  )
40
41  FILTER (
42    ?isCalorieConscious = false ||
43    (BOUND(?mealCalories) && xsd:decimal(?mealCalories) < xsd:decimal(?maxCalories))
44  )
45
46  FILTER (
47    LCASE(STR(?diet)) != "vegan" || (?isVegan = true)
48  )
49  FILTER (
50    LCASE(STR(?diet)) != "celiac" || (?notContainsGluten = true)
51  )
52  FILTER (
53    LCASE(STR(?diet)) != "carnivore" || EXISTS { ?meal per:hasIngredient per:meat }
54  )
55 }

```

Figure 4.3: SPARQL Query for Personalized Meal Recommendation

We now present several test cases to illustrate how the system behaves under different user inputs. Each example shows both the user-defined parameters and the corresponding output.

Example 1: Main Dishes, Omnivore, no allergies, no calorie restriction

ID: RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10
Instantiation Type: Instance

Relation	Value	Actions
hasCoursepreference	Main Dishes	<button>Remove</button>
hasAllergy	none	<button>Remove</button>
hasDiet	Omnivore	<button>Remove</button>
isCalorieConscious	false	<button>Remove</button>
maxCalorieIntake	0	<button>Remove</button>

▼

Add Relation

Save

Close

Figure 4.4: Example 1 Input Parameters

- 1 LasagnaAllaBolognese
- 2 PenneAllArrabbiata
- 3 RisottoAiFunghi
- 4 SpaghettiAllaCarbonara

Figure 4.5: Example 1 Meal Recommendation Output

Example 2: Pizza, Carnivore, no allergies, no calorie restriction

ID: RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10

Instantiation Type: Instance

Relation	Value	Actions
hasCoursepreferenc	Pizza	<button>Remove</button>
hasAllergy	none	<button>Remove</button>
hasDiet	Carnivore	<button>Remove</button>
isCalorieConscious	false	<button>Remove</button>
maxCalorieIntake	0	<button>Remove</button>

▼ Add Relation

Save Close

Figure 4.6: Example 2 Input Parameters

- 1 Diavola
- 2 ProsciuttoEFunghi

Figure 4.7: Example 2 Meal Recommendation Output

Example 3: Second Course, Celiac, allergic to Cereals, max 700 kcal

ID: RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10

Instantiation Type: Instance

Relation	Value	Actions
hasCoursepreference	Second Course	Remove
hasAllergy	cereals	Remove
hasDiet	Celiac	Remove
isCalorieConscious	true	Remove
maxCalorieIntake	700	Remove

Add Relation

Save

Close

Figure 4.8: Example 3 Input Parameters

- 1 BisteccaAllaFiorentina
- 2 PolloAllaCacciatora

Figure 4.9: Example 3 Meal Recommendation Output

Example 4: Dessert, Vegan, allergic to Nuts, max 600 kcal

ID: RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10

Instantiation Type: Instance

Relation	Value	Actions
hasCoursepreferenc	Dessert	Remove
hasAllergy	nuts	Remove
hasDiet	Vegan	Remove
isCalorieConscious	true	Remove
maxCalorieIntake	600	Remove

▼

Add Relation

Figure 4.10: Example 4 Input Parameters

1 Biscotti

Figure 4.11: Example 4 Meal Recommendation Output

Example 5: Main Dishes, Carnivore, allergic to Eggs, max 800 kcal

ID: RecommendMeals_4d13746c-0df8-47d3-a84b-5e069cc75b10
Instantiation Type: Instance

Relation	Value	Actions
hasCoursepreference	Main Dishes	<button>Remove</button>
hasAllergy	eggs	<button>Remove</button>
hasDiet	carnivore	<button>Remove</button>
isCalorieConscious	true	<button>Remove</button>
maxCalorieIntake	800	<button>Remove</button>

Add Relation

SaveClose

Figure 4.12: Example 5 Input Parameters

- 1 LasagnaAllaBolognese
- 2 PenneAllArrabbiata
- 3 RisottoAiFunghi

Figure 4.13: Example 5 Meal Recommendation Output

5. Conclusions

5.1 Lolli Andrea

This project offered an insightful exploration into the design and development of a personalized menu recommendation system, highlighting both the potential and the limitations of applying such technology within the context of QR code-based menus. But before diving into the analysis of the various solutions, I would like to share some thoughts and reflections that arose during the course of this project. While they may not be entirely relevant to the core discussion, I hope they will inspire in the reader the same curiosity and interest they sparked in me.

Personal Reflections and Initial Considerations

In the early stages, the idea of representing meals entirely as detailed lists of ingredients was considered. However, the final approach prioritized key factors like allergen presence and dietary restrictions rather than exhaustive ingredient lists. This decision stems from the recognition that not all ingredients are equally relevant to recommendation filtering; for example, allergens identified under EU Regulation 1169/11 provide a practical and legally grounded basis for exclusion. Furthermore, categorizing meals by dietary profiles such as carnivore or vegetarian enhances the system's adaptability without unnecessary complexity. Calorie management also influenced our design choices. Calculating calories by summing individual ingredient values proved unreliable due to varying portion sizes and cooking methods. Instead, associating calorie information with the complete dish as provided by the restaurant ensures more accurate and consistent guidance for calorie-conscious diners.

Another reflection concerns the constraints of QR code menus and typical diner behaviour. Although personalized meal suggestions can streamline decision-making, many users prefer to browse the full menu to explore all available options before finalizing their choice. Thus, a system that solely presents filtered recommendations risks limiting user autonomy. To balance personalization with freedom of choice, an optimal design should emphasize providing both a curated list of suitable meals and full access to the entire menu.

A significant limitation of the current system is the lack of persistent user data storage. While this design choice enhances user privacy by requiring preference input on every visit, it also constrains the user experience by introducing repetitive input tasks. An improved version could incorporate anonymous preference storage—either locally or in compliance with privacy regulations on secure servers—to allow seamless personalization across multiple visits. Additionally, integrating with widely used restaurant

platforms could enable users to maintain a unified dietary profile across venues, further enhancing convenience and adoption.

However, while this project may not be immediately practical for typical QR code menus, since users generally prefer to browse full menus without extra input, it provided a valuable introduction to knowledge engineering. Coming from a traditional programming background, I initially thought a simple database-driven solution would suffice, but I soon realized the importance of modeling knowledge and information in this context. This experience offered a fresh perspective on solving knowledge-intensive problems and served as an excellent foundation for exploring personalized recommendation systems, even if the project itself isn't directly viable for restaurant customers yet.

Technical Analysis of Implemented Solutions

Several technical approaches were explored during this project to implement the Personalized Menu System, each presenting unique advantages and challenges. The following analysis reflects on my experience with each solution and compares their usability, maintainability, and alignment with knowledge engineering principles.

DMN (Trisotech and Camunda)

The DMN-based solutions developed using Trisotech and Camunda were notable for their clarity and ease of use. These platforms made it straightforward to represent and update restaurant menus, which is essential for practical deployment. The use of enumerated datatypes within decision tables helped prevent input errors, ensuring system reliability. Additionally, these solutions facilitated easy onboarding for restaurant owners by allowing menu import from scratch. However, our Trisotech solution tended to be more algorithmic, relying heavily on FEEL regular expressions for filtering, which somewhat limited its focus on pure knowledge representation. In comparison, the earlier Camunda solution integrated meal definitions and FEEL expressions directly inside decision tables, better reflecting the knowledge-driven nature of the project. Thus, while DMN offers a practical and accessible approach, its expressiveness in capturing complex relationships is somewhat limited.

Prolog

Working with Prolog was particularly rewarding. Prolog's strength in representing complex relationships and logic made it well-suited for capturing the necessary domain knowledge. Our Prolog implementation approached an ontology-like structure by modeling relationships among ingredients, allergens, ingredient types, diets, and meals, resulting in a highly expressive system aligned with knowledge engineering goals. Moreover, the modularity of the Prolog code facilitated ease of maintenance and extension, allowing new properties to be added with minimal effort. This flexibility and clarity highlight Prolog as a robust option for knowledge-intensive applications.

Ontology-Based Approach

The ontology solution offered the most faithful representation of the domain knowledge, explicitly modeling connections between meals, ingredients, allergens, and dietary preferences. This provides a clear, formal understanding of the system's components. However, developing and maintaining a coherent ontology proved challenging and labor-

intensive. Significant effort was required for initial design, and updates demand careful management to maintain consistency. Despite these challenges, the ontology's ability to formally capture domain knowledge makes it a powerful tool.

Agile and Ontology-based Meta-modelling

The AOAME tool provided a unique combination of ontology management and business process modeling. Although initial setup presented technical difficulties, the experience became enjoyable once the tool was mastered. AOAME offered better means for maintaining, utilizing, and expanding the ontology compared to stand-alone editors. A key advantage was its support for BPMN 2.0, which allowed visualization of business processes underlying the system, bridging domain knowledge with operational workflows. This feature improves stakeholder communication and understanding, making AOAME valuable for projects requiring both knowledge representation and process management.

In summary, each solution contributed valuable insights. DMN platforms provided practical and user-friendly decision logic implementations but with some expressiveness limitations. Prolog excelled in semantic richness and modularity, making it ideal for knowledge-focused systems. Ontologies captured comprehensive domain knowledge but with increased complexity in development and maintenance. Finally, AOAME combined ontology and process management effectively, enhancing both knowledge handling and operational clarity.

5.2 Berdini Angelica

In this chapter, I present my final reflections on the project, organized around the different technologies and methodologies that were explored. The focus is not only on comparing their strengths and weaknesses but also on examining the critical differences and opportunities that emerged when applying multiple approaches to the same problem. Beyond simply choosing a technology, this process highlighted the importance of understanding which reasoning model best fits the context and objectives of the project. Sometimes, the most expressive or sophisticated solution is not necessarily the most appropriate, a simpler approach can be more effective when it aligns better with user expectations and practical constraints.

DMN

This approach turned out to be intuitive and relatively easy to implement thanks to tools such as Camunda and Trisotech. The main advantage is the clarity of rule definition and the fact that a domain expert can easily understand and maintain them. However, I encountered expressiveness limitations, especially in handling more complex conditions and lists (such as allergen checks), which required alternative solutions or model adjustments. The transition from Camunda to Trisotech initially required some effort, but the result was very satisfying. Not only did I gain a deeper understanding of the underlying logic, but I also became much more familiar with FEEL expressions. Although it was ultimately the least intuitive approach for developing the allergen filtering logic, the experience was highly rewarding and laid the groundwork for understanding how to develop the Prolog and Ontology solutions more effectively.

Prolog

Thanks to the time dedicated to understanding the project's goals and structure, as well as to the development carried out in Trisotech, Prolog proved to be a highly versatile and modular tool. Given the familiarity of the language for those with a computer science background, it may not be the most intuitive platform, but it was certainly the most straightforward to use. This tool allowed me to think of a lot of extensions and develop them quickly. In the end, it allowed me to gain a deeper understanding of the underlying logic and to experiment more effectively.

Ontology-based Approach

Using ontologies and knowledge graphs was the most formal and semantic approach. I really liked how reusable, clear, and modular the development process was. At first, I found working with ontologies harder compared to the other tools, especially since converting between DMN and Prolog felt like a simpler extension, while with ontologies I had to look at the problem from a different angle.

But once I designed the ontology and set the rules, testing became easier. I also appreciated how the system could automatically infer many properties and how flexible it was to expand and adapt to different scenarios. Out of all the tools I used, ontologies definitely took the most time for analyzing the problem, and I had to redefine parts of it several times.

In the end, even though it was challenging, the powerful reasoning and flexibility made it a very rewarding approach.

Agile and Ontology-based Meta-modelling

At first, AOAME tool was a bit challenging to understand, but once I understood its full potential, the experience improved significantly and led to many extensions of the ontology with minimal effort. Accessing and expanding the ontology this way was really convenient because AOAME provided a solid foundation to build upon. It greatly reduced complexity, helped me better understand the process and structure, and made extending the ontology much easier.

Moreover, using Jena Fuseki allowed for complex yet intuitive queries that were easy to test. In contrast, performing similar queries in Protégé was more difficult and sometimes even impossible. The expressiveness I achieved with Jena Fuseki was better than that of Protégé, making it a powerful tool for advanced querying and reasoning.

Personal Reflections

The most interesting part of the project was the continuous shift in perspective. Starting from the same problem, the solutions developed were different but effective in every case. However, they truly required understanding how the logic behind each approach varies. I like to take the example of developing ingredients and allergens, each approach was unique, presented its own challenges, and demanded multiple trials before reaching a satisfactory result. This made me constantly ask myself questions like: Is this really what I want to achieve? Is it more logical to have categories of ingredients or individual ingredients? What types are they? How would a user or a restaurant input them?

These and many other questions throughout the project pushed me to explore deeper and experiment, always wondering: what is the right solution? The answer I found is that there is no single best solution. Each approach has its pros and cons. DMN was fun and intuitive to develop but somewhat limited. Prolog was familiar but still required a shift in thinking. Ontologies were challenging at first, yet the reasoning process turned out to be very straightforward.

It was also very interesting to use AOAME and Jena Fuseki, seeing how the work done in the ontology could be reused and queried much more easily compared to Protégé made me question whether the choices made in the ontology were sound. Throughout the development, the solution chosen was not always the most optimal technically, but rather a compromise between what the tool allows and is designed for, the problem at that time and its potential use by clients.

This project will definitely help me in the future to carefully select the approach and methodologies for representing knowledge based on utility, scenarios, users, and more. In conclusion, it was fascinating to put into practice the pros and cons of these technologies.